

# Pruebas de software

---

- La prueba consiste en la ejecución de un programa con el único objetivo de encontrar defectos. Las condiciones bajo las cuales se realizará la prueba tienen que estar prefijadas para poder evaluar los resultados que obtengamos.
- Es decir, las pruebas no se hacen al azar, sino que seguirán una estrategia. En ella será necesario definir una serie de casos de prueba que son un conjunto de entradas, de condiciones de ejecución y resultados esperados, desarrollados para un objetivo particular.

# Pruebas de software

---

- La calidad de una aplicación o software no es sólo que el software funcione bien y vaya rápido, implica además que satisfaga las expectativas de los clientes, usabilidad, accesibilidad, confiabilidad, coste, eficiencia y cumplimiento de los estándares.
- Para asegurar la calidad, debemos hacer uso de las pruebas de software.
- Estas pruebas consumen tiempo y recursos, pero son fundamentales para que los fallos en la aplicación sean mínimos.
- Es imposible que los desarrolladores sepamos de antemano de forma total y absoluta, las necesidades de los clientes, es por ello, que como fase final de la estrategia de pruebas, el software termine siendo usado por los clientes durante un periodo de prueba.

# Objetivos de las pruebas de software

---

- El objetivo de las pruebas de software es encontrar la mayor cantidad posible de errores, de forma que el software tenga el mínimo posible de errores.
- No sólo basta con ejecutar código y que funcione, sino que hay que hacerlo mediante requisitos, especificaciones funcionales, diseños, etc.
- Las pruebas deben entenderse como:
  - Verificación: el producto que se está construyendo funciona correctamente; es decir, es capaz de realizar la tarea para la cual ha sido diseñado.
  - Validación: el producto terminado, además de ser correcto, es conforme con lo que se el cliente había esperado.

# Importancia de las pruebas

---

- La realización de pruebas contribuye a:
  - mejorar la satisfacción del cliente
  - incrementar la productividad
  - disminuir los riesgos en el negocio
  - facilitar el crecimiento del negocio

# Estrategia de pruebas

---

- La estrategia de pruebas será la que mejor se adecúe a nuestro proyecto.
- Todos los desarrollos de software tienen una característica en común: lo desarrollan personas. Las personas cometemos errores y cambiamos de ideas, por eso hay que probar y volver a probar.
- En general, la estrategia de pruebas consta de 3 partes:
  1. **Visión general de la estrategia de pruebas:** se definen las pruebas que se van a realizar, diferentes versiones de las pruebas y **objetivo que se pretende alcanzar** con cada una de ellas.
  2. **Tareas en la realización de las pruebas:** **descripción de las tareas propias a realizar** y **lugar de realización de las mismas**, **datos de entrada de las pruebas** y **finalización de las mismas**.
  3. **Resultados:** consiste en **documentar y revisar los resultados esperados y los reales**. Si un caso de prueba falla, **debe quedar reflejado el defecto detectado**.

# Limitaciones del proceso de pruebas

---

- En la práctica se considera que el número de pruebas es teóricamente infinito. Pero en algún momento hay que parar. El número de pruebas a realizar implicará un equilibrio entre recursos y tiempo.
- Por muy cuidadosos que seamos en la estrategia de pruebas, cuando la aplicación ya esté en uso por el cliente, éste detectará errores y/o funcionamiento que no es el esperado. Así que deberemos revisar la aplicación y corregir estos errores.

# Índice

---

- Pruebas de software
- **Tipos de pruebas**
- Pruebas automáticas
- Pruebas en React

# Tipos de pruebas (I)

---

➤ Según la bibliografía que se consulte, encontraremos unos u otros tipos de pruebas. En general, podemos clasificarlas en:

- **Pruebas unitarias:** se realizan a módulos o clases del programa, por separado. Es decir, a cada unidad que compone la aplicación.
- **Pruebas de integración:** integran componentes y módulos del programa según un orden preestablecido. Se prueba el funcionamiento de la interacción de los componentes del software.
- **Pruebas de sistema:** verifican el comportamiento del software en diferentes sistemas operativos o interaccionando con diferente hardware (son más comunes en software de escritorio).



# Tipos de pruebas (II)

---

- **Pruebas de regresión:** después de realizar algún tipo de modificación en el código del programa, consisten en volver a ejecutar un conjunto de pruebas que se han llevado a cabo anteriormente para asegurarse de que los cambios no han propagado efectos colaterales no deseados.
- **Pruebas funcionales:** su objetivo es detectar errores en la implementación de los requerimientos. Es este tipo de pruebas no interesa el código, ni el algoritmo, ni la eficiencia ni la construcción de elementos innecesarios. Sólo interesa verificar que las salidas que devuelve la aplicación son las esperadas, en función de las entradas que tenga.
- **Pruebas de capacidad y rendimiento.** La prueba de capacidad (también conocida como prueba de resistencia) ejecuta un sistema de forma que demande recursos en cantidad, frecuencia o volúmenes anormales. La de rendimiento determina los tiempos de respuesta (lo rápido que realiza una tarea un sistema en condiciones de trabajo), el espacio que ocupan los módulos en disco y en memoria, el flujo de datos que se genera, etc. Estas son pruebas típicas de aplicaciones de escritorio.

## Tipos de pruebas (III)

---

- **Pruebas de seguridad:** la prueba de seguridad intenta verificar que los mecanismos de protección incorporados en nuestro sistema lo protegen de accesos no autorizados.
- **Pruebas de usuario:** este tipo de prueba (también conocida como de usabilidad) se refiere a asegurar que la interfaz de usuario (GUI) sea amigable, intuitiva y que funcione correctamente.
- **Pruebas de aceptación:** estas pruebas son realizadas por el cliente final. Consisten en pruebas funcionales sobre el sistema completo. Se realizan sobre el producto terminado y nunca durante el desarrollo del sistema o de la aplicación.

# Índice

---

- Pruebas de software
- Tipos de pruebas
- **Pruebas automáticas**
- Pruebas en React

# Pruebas automáticas

---

- Todos los tipos de pruebas que estamos viendo se ejecutan de manera regular con el objetivo de probar que la evolución del software desarrollado funciona perfectamente, tanto para las funcionalidades nuevas como para aquellas que ya existían. Una de las claves es que las pruebas sean automatizadas, porque si se ejecutaran manualmente, el coste de hacerlo "continuamente" sería inasumible.
- Es importante, en el caso de usar herramientas que automaticen el proceso de pruebas, realizar una correcta elección de la misma.
- La elección de la herramienta de automatización de pruebas depende del tipo de pruebas a automatizar, el entorno de desarrollo usado y del lenguaje de programación con el que hayamos codificado la aplicación.

# Ejemplos de herramientas para automatizar pruebas

| Tipo de prueba           | Herramienta/Librerías | Lenguaje                 |
|--------------------------|-----------------------|--------------------------|
| Unitarias                | JUnit                 | Java                     |
|                          | PHP Unit              | PHP                      |
|                          | Jest/Vitest           | React                    |
| De integración           | TestNG                | Java                     |
|                          | React Testing Library | React                    |
| Funcional y de regresión | Selenium              | Java, PHP, Python, React |
| Capacidad y rendimiento  | JMeter, Jcrawler      | Java                     |
| Aceptación               | Concordion, FitNesse  | Java, Python, .NET       |